# Pathlight Tracker

A Deep-Dive Theoretical & Technical Guide: Mental Models, Architecture, Tech Stack, and Code Progress

Architecture: Local Multi-Process | Monorepo: pnpm + Turborepo | Current Phase: Phase 0 (Setup & Fixtures)

> **Mentor Overview**
>
> Welcome to the Pathlight Tracker architectural guide. This document explains the philosophy, design choices, data flow, code instrumentation techniques, and foundational codebase constructed so far.

## 1. The Mental Model: Why Pathlight Tracker Exists

When diagnosing unexpected bugs, performance issues, or incorrect data transformations in backend applications (Node.js / Express / NestJS), developers typically rely on two suboptimal techniques:

- **Scattering `console.log` statements:** Slow, noisy, requires manual code modifications, and litters terminals with unstructured output that must be manually cleaned up later.
- **Traditional Step Debuggers:** Require setting breakpoints, pausing execution, and stepping through voluminous framework and library internals, breaking natural request concurrency.

**Pathlight Tracker** is a **local-first runtime visualizer**. It hooks transparently into your Node.js application at startup. The moment an HTTP request arrives, Pathlight Tracker records the full path the request travels through:

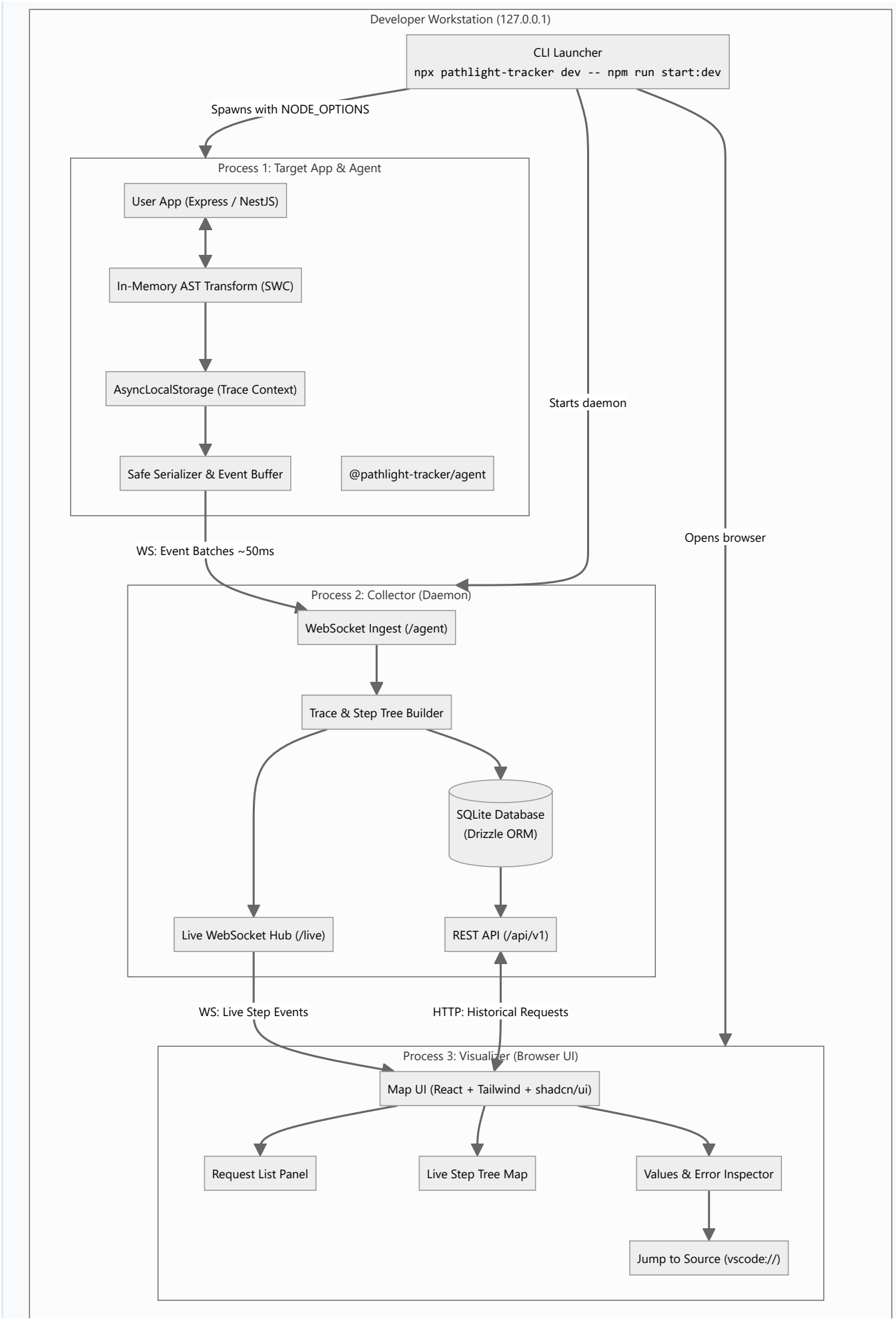
- Every function entered and exited, with exact arguments and return values.
- Framework lifecycle events: Guards, Middleware, Pipes, Interceptors, and Controllers.
- Outbound database queries (TypeORM, Prisma, Mongoose) and HTTP requests.
- Condition decisions ( `if` / `else if` / `switch` branches taken).
- Handled and unhandled exceptions (highlighting the exact failure line in red).

> **Zero-Cloud, Zero-Telemetry Privacy Guarantee**
>
> Pathlight Tracker binds strictly to `127.0.0.1`. No data, logs, traces, or metadata are ever transmitted outside the developer's workstation.

## 2. System Architecture: The Three Cooperating Processes

Pathlight Tracker operates as three distinct, lightweight processes coordinated on your local machine:

FIGURE 1: HIGH-LEVEL SYSTEM ARCHITECTURE & PROCESS BOUNDARIES


Developer Workstation (127.0.0.1)
CLI Launcher
npx pathlight-tracker dev -- npm run start:dev
Spawns with NODE_OPTIONS
Starts daemon
Opens browser
Process 1: Target App & Agent
User App (Express / NestJS)
In-Memory AST Transform (SWC)
AsyncLocalStorage (Trace Context)
Safe Serializer & Event Buffer
@pathlight-tracker/agent
WS: Event Batches ~50ms
Process 2: Collector (Daemon)
WebSocket Ingest (/agent)
Trace & Step Tree Builder
SQLite Database
(Drizzle ORM)
Live WebSocket Hub (/live)
REST API (/api/v1)
WS: Live Step Events
HTTP: Historical Requests
Process 3: Visualizer (Browser UI)
Map UI (React + Tailwind + shadcn/ui)
Request List Panel
Live Step Tree Map
Values & Error Inspector
Jump to Source (vscode://)

### Process 1: The Agent (Records)

Preloaded via `NODE_OPTIONS="--require @pathlight-tracker/agent"`. Intercepts requires, wraps user functions in-memory, tracks async context, serializes parameters safely, and pushes event batches.

### Process 2: The Collector (Thinks)

Fastify server. Receives events, validates schemas using Zod, stitches steps into hierarchical trees, records to SQLite, and broadcasts live updates.

### Process 3: The Map UI (Shows)

Browser React application. Renders real-time execution graphs ("steps", not "spans"), provides inspectors for complex payloads, and links directly to VS Code source code lines.

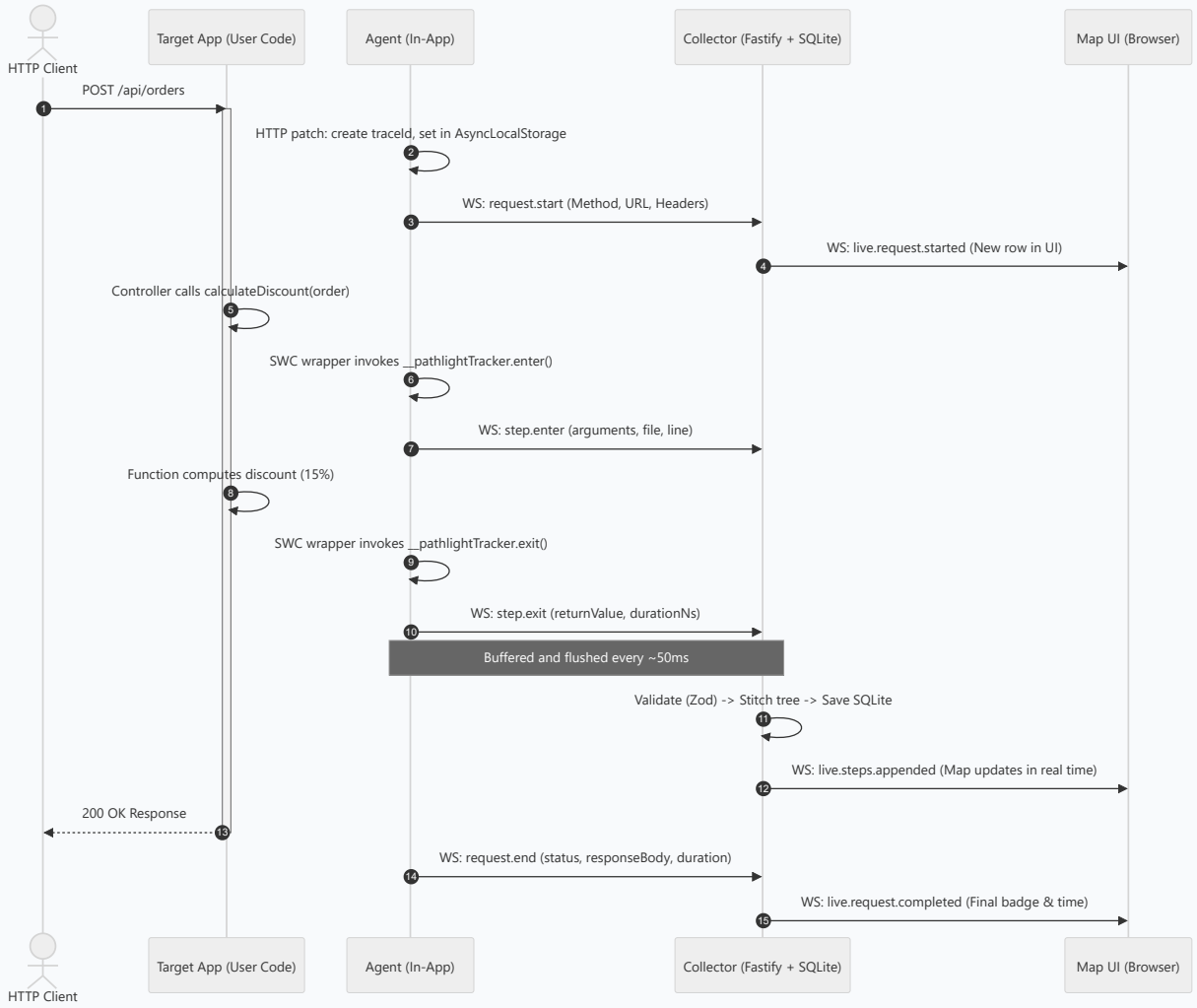
### The Host-Agnostic Core

The UI and Collector are designed to run in a standalone browser tab today, and seamlessly inside a VS Code webview panel in Phase 2 with zero core rewrites.

# 3. End-to-End Request Lifecycle

Tracing an HTTP request through every layer from ingress to visual rendering:

FIGURE 2: SEQUENCE OF A TRACED REQUEST ACROSS LAYERS

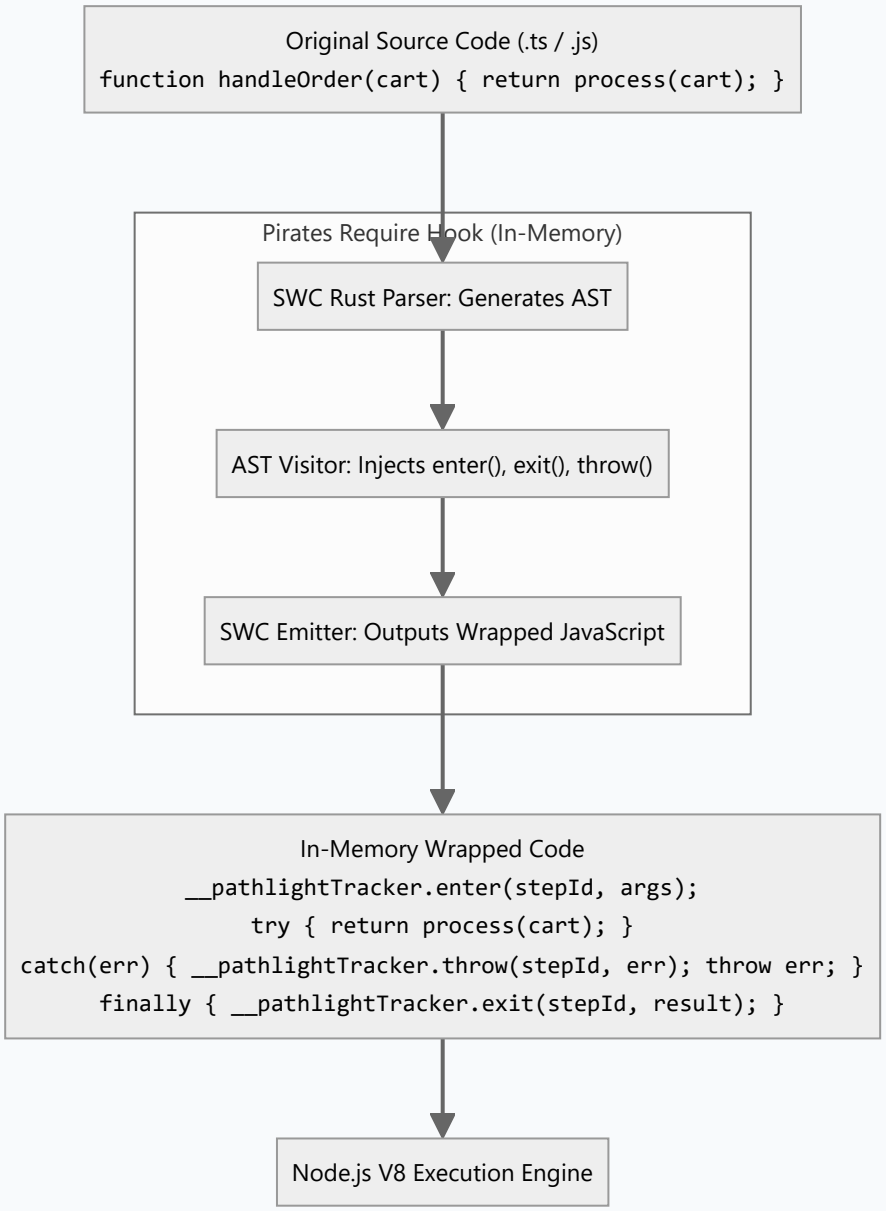



## Key Steps in the Request Journey:

1. **Inbound Capture:** The Node `http` patch intercepts the incoming socket, creates a nanosecond-precision timestamp and prefixed ULID ( `req_...` ), and populates `AsyncLocalStorage` .
2. **Framework Lifecycle:** NestJS and Express patches emit intermediate steps for route guards, pipes, and middleware before user code runs.
3. **User Function Execution:** Every instrumented function triggers `enter` and `exit` / `throw` hooks.
4. **Batching & Transport:** Events are enqueued in a ring buffer and dispatched over a persistent WebSocket in ~50 ms batches, reducing network overhead.
5. **Live Tree Construction:** The Collector validates incoming payloads with Zod, connects child steps to parent steps using context stacks, and writes to SQLite.

# 4. Under the Hood: AST Transformation & Async Context

FIGURE 3: IN-MEMORY AST INSTRUMENTATION FLOW




## Core Agent Principles:

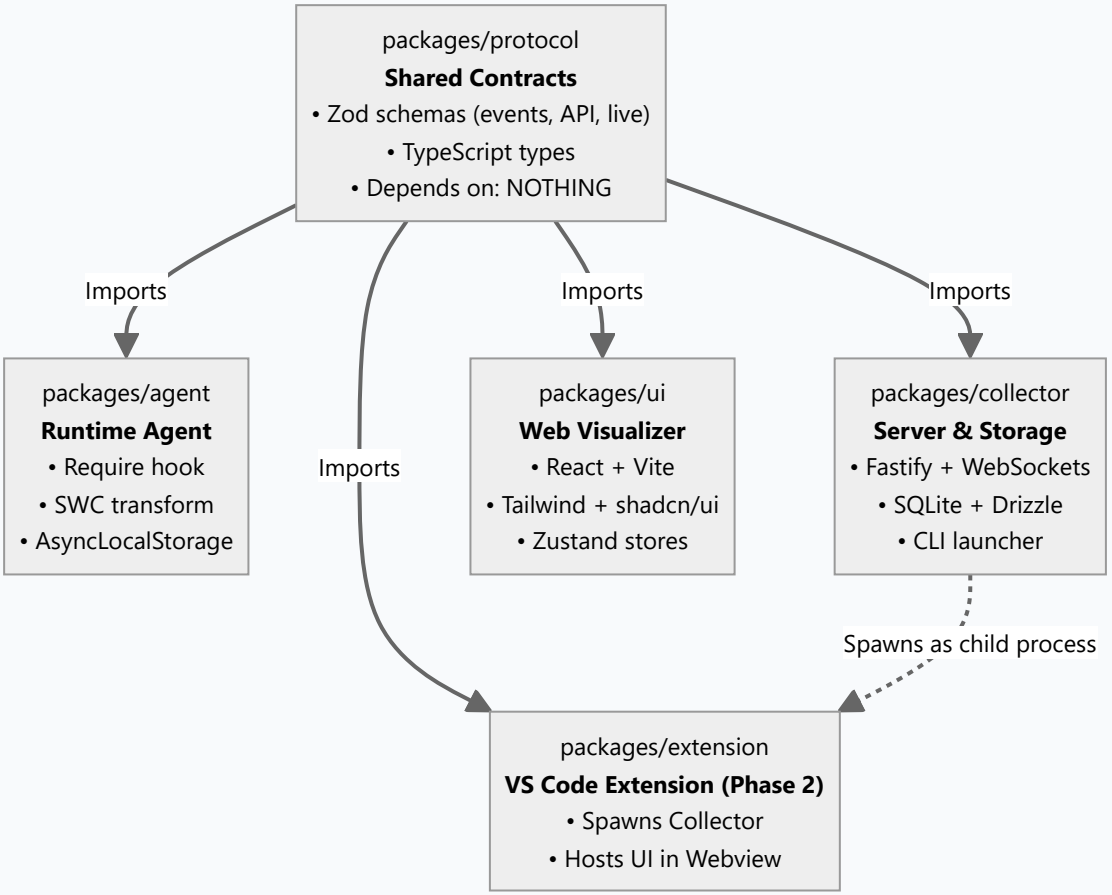
- **"The User's App Always Wins":** The agent is wrapped in strict try-catch handlers. If the agent encounters an internal serialization error or cannot reach the collector, it fails silently (*fails open*). It must never throw an error into user application logic.
- **In-Memory Transpilation:** Files on disk are never altered. The require hook (using `pirates`) intercepts Node's module compilation in memory via SWC.
- **AsyncLocalStorage (ALS):** Handles asynchronous concurrency. Node processes thousands of concurrent promises. ALS guarantees that callbacks and async/await steps remain paired with their exact

originating `traceId`.

- **Safe Serialization & Redaction:** Prevents circular reference crashes, limits string previews to 500 characters, truncates deep objects, and redacts sensitive keys (e.g., `password`, `token`, `authorization`).

## 5. Monorepo Architecture & Dependency Boundaries

FIGURE 4: MONOREPO PACKAGE DEPENDENCIES & STRICT BOUNDARIES




**Strict Boundary Rule**

`protocol` depends on nothing. `agent`, `collector`, and `ui` depend *only* on `protocol` and never on each other! This ensures clean isolation, prevents circular dependencies, and allows individual packages to be tested and built in isolation.

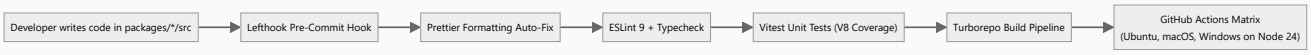
## 6. Technology Stack Rationale & Advantages

| Technology | Role in Pathlight Tracker | Why It Was Chosen & Advantages |
|---|---|---|
| **pnpm (v10)** | Package manager & workspace manager | Hard-link store saves disk space; strict isolation prevents phantom dependencies (undeclared imports fail immediately). |
| **Turborepo (v2)** | Build & task orchestrator | Topological execution graph ( `^build` ); hashes input files and caches build/test artifacts to eliminate redundant work. |
| **TypeScript Strict** | Type system across all packages | `strict: true` with ES2023 language features and CommonJS compilation targets for seamless Node `--require` compatibility. |
| **Vitest (v5)** | Unit test runner & code coverage | Native ESM and TS execution; instant startup; integrated V8 coverage engine ( `@vitest/coverage-v8` ). |
| **ESLint 9 + Prettier 3** | Code quality & formatting | Modern flat config ( `eslint.config.mjs` ) with type-aware linting; Prettier ensures consistent code formatting across all packages. |
| **Lefthook** | Git pre-commit hooks | Fast Go-based hook runner; checks and formats staged files before commits occur, ensuring zero broken commits reach git. |
| **Fastify** | Collector HTTP & WebSocket daemon | High-throughput, low-overhead event processing; robust plugin ecosystem; ideal for streaming live execution graphs. |
| **SQLite + Drizzle** | Local trace store | Single-file database ( `.pathlight-tracker/tracker.db` ); zero infrastructure or Docker daemon needed on developer machines. |
| **SWC (Rust)** | Code AST transformation engine | 20x faster than Babel; performs AST parsing and function wrapping in memory with imperceptible cold-start delay. |
| **React + shadcn/ui** | Live Map Web UI | Component-based UI with accessible primitives and design token isolation (light/dark mode). |

# 7. What We Have Built In Code (Phase 0 Progress)

We have established the core monorepo infrastructure, developer ergonomics, automated quality gates, and cross-platform continuous integration:

FIGURE 5: PHASE 0 VERIFICATION & QUALITY PIPELINE




## Detailed Breakdown of Implemented Tasks:

- **Task 0.1 · Validate Against AppMap (Done):** Conducted market and technical research. Validated architectural advantages: live streaming vs file-based recording, full runtime variable capture, and dedicated support for Express and NestJS.
- **Task 0.2 · Create the Monorepo (Done):** Initialized pnpm workspace with Turborepo, `tsconfig.base.json`, and created package shells: `packages/protocol`, `packages/agent`, `packages/collector`, and `packages/ui`.
- **Task 0.3 · Linting & Formatting (Done):** Configured ESLint 9 flat configuration with `typescript-eslint` v8, Prettier 3, and Lefthook pre-commit git hooks with automatic re-staging (`stage_fixed: true`).
- **Task 0.4 · Testing Setup (Done):** Integrated Vitest with a root workspace configuration, colocated unit test structure (`*.test.ts`), and 100% initial code coverage via `@vitest/coverage-v8`.
- **Task 0.5 · Continuous Integration (Done):** Created GitHub Actions matrix testing clean builds across Ubuntu, macOS, and Windows on Node 24 with `--frozen-lockfile` verification, and initialized Changesets for automated versioning.
- **Task 0.6 · Fixture Apps (In Progress):** Setting up standalone Express and NestJS backend test apps to benchmark real-world tracing, test endpoint responses, and verify deliberate 500 error capture.

## 8. The Roadmap Ahead: Milestones to MVP

| Phase | Key Deliverables | Success Verification |
| --- | --- | --- |
| **Phase 1: Capture Spike** | Require hook + SWC transform spike, AsyncLocalStorage context, and performance benchmark. | Verify that instrumented code preserves 100% of original application behavior with less than 5% latency overhead. |
| **Phase 2: Protocol & Collector** | Event schemas, Fastify ingestion server, Drizzle SQLite schema, REST API, and live WebSocket hub. | Collector ingests 10,000 events/second, persists to SQLite, and pushes to connected UI clients. |
| **Phase 3: Runtime Agent** | Express and NestJS patches, database boundary nodes (TypeORM, Prisma, Mongoose), redaction, and fail-open guards. | Agent automatically captures call trees and values for requests in both fixture apps without changing behavior. |
| **Phase 4: Map UI** | React map layout, live step tree, variable inspector, error highlighting, and editor jump links. | User views real-time request maps in the browser and clicks any step to open the exact line in VS Code. |
| **Phase 5: Release 0.1.0** | End-to-end integration tests, doctor CLI command, security audit, and packaging. | Single-command developer launch: `npx pathlight-tracker dev -- npm run start:dev`. |

Generated autonomously for Pathlight Tracker • Single source of truth: `Docs/tracer-architecture.md` • All rights reserved.